

CIA triad analysis of a Secure Authentication System

1. System Context

The system being analysed is a secure authentication system I developed using high-level technologies such as Python and SQLite. The system allows for users to create accounts, users to create, view and delete their own data, and features user authentication in the form of logins. The system handles data such as login attempt logs, usernames, password hashes, password salts and encrypted user notes. The system will be used by authenticated users accessing their own data and administrators will likely use it to monitor logs. This system handles highly sensitive authentication data which if leaked could lead to data leaks and account abuse. If this was to happen, then the system would lose all integrity and trust. This is why security is the absolute priority of this system. Finally, this system is relevant to real-world applications in many ways such as financial applications which need to handle sensitive user credentials making security a critical design consideration.

2. Confidentiality

Sensitive data identified

Confidentiality refers to the practice of ensuring that sensitive data can only be accessed by authorised users and is prevented from unauthorised disclosure. In this system, the most sensitive data consists of password hashes, salts, encrypted user notes and authentication logs. If any of this data is leaked, then the consequences could include unauthorised access to accounts, a loss of trust in the system and malicious activity such as the deletion of data on the system

Controls implemented

To protect sensitive information, data such as passwords and notes are not stored in plaintext. Instead, passwords are hashed using a hashing algorithm using a unique salt for each user, ensuring that even if the data was compromised attackers would never be able to recover exact user passwords. Additionally, the use of unique salts prevents attackers from using hash tables to target multiple accounts.

User notes are stored exclusively in their encrypted form via symmetric encryption, and when a user wants to view the note, it is decrypted and then printed. User notes are never stored in plaintext in the database to prevent them from being compromised in the event of a data leak as long as the key remains secure.

When a user goes to view their notes after authentication, access controls have been used to ensure that users cannot see other users' data without their credentials. If these measures were not put in place, users would be able to see each other's private information potentially leading to attacks from users within the database.

To mitigate SQL injection attacks, parameterised SQL statements have been used when handling the database. This prevents attackers from modifying SQL statements with malicious intent by entering SQL code when prompted to enter credentials. If SQL injection attacks were not mitigated, then attackers could easily use these attacks to gain unauthorised access to the system and potentially delete and modify existing user information.

Weaknesses / Trade-offs

Whilst these control measures significantly improve confidentiality, there are limitations to this system. Firstly, the encryption key is currently being stored alongside application code. This could allow an attacker with server access to decrypt stored data, highlighting a trade-off between implementation simplicity and stronger confidentiality guarantees surrounding the importance of secure key management in real-world systems.

3. Integrity

Data that requires integrity

Integrity refers to the practice of ensuring that data remains accurate, consistent, and protected from unauthorised modification. This system contains data such as authentication logs, user credentials and encrypted notes which must remain trustworthy, to ensure accurate system behaviour and maintain user confidence.

Controls implemented

To ensure that data retains its integrity, parameterised SQL queries are used when interacting with the database. This helps mitigate attackers inputting SQL code when prompted to input credentials to modify or delete existing data, as it treats user inputs solely as data instead of executable commands.

Access controls are enforced through the SQL code used to view, create and delete data, by ensuring that the data is associated with the authorised user's user ID. This prevents users from viewing and deleting notes that belong to other users even if they have successfully authenticated. Because of this, the integrity of user-specific data is maintained.

The system will also log all authentication attempts. System administrators can use this log to identify suspicious activity such as failed login attempts. Whilst this will not prevent any direct attempt to modify data, it supports integrity by allowing for easy detection of abnormal behaviour that may indicate malicious intent.

Limitations & Trade-offs

Firstly, if an attacker gains access to the database, then logs could be modified or deleted. This is because the logs have no security measures like being read-only or verification mechanisms. It also does not alert system admins immediately when suspicious activity is happening, instead they must read through logs to find malicious intent. This highlights a large trade-off of implementation simplicity and stronger integrity guarantees that would be needed in a production environment.

4. Availability

What needs to be available

Availability is the practise of ensuring legitimate users can access the system and the system's data when required. In this system, availability is needed to allow users to authenticate, access their notes and perform authorised actions without disruption.

Threats to availability

This system has a few weaknesses regarding availability. Firstly, the lockout system works via username, therefore attackers can create bots to repeatedly guess passwords ensuring that users can never gain access to their accounts via abusing the lockout system. Secondly, by using SQLite, the database has performance limits making it not suitable for high scale usage. This means that the database will be susceptible to denial-of-service attacks if attackers create a bot to repeatedly send requests to the server faster than it can process them. Overall, these factors could lead to a failure in availability and loss of trust in the system by users.

Controls implemented

To support availability whilst mitigating attacks, the system keeps track of all authentication attempts via a logging system and will prevent users from authenticating if they have had too many failed attempts. However, the lockout duration is not too long, so users will be able to attempt authentication again after a short period of time. This helps prevent attackers from using brute-force attacks to guess existing user's passwords and gain unauthorised access to their data.

A logging system is used to track all authentication attempts to gain access to the system. This can be used by system administrators to attempt to find malicious intent to reduce availability such as repeated requests sent to the server to produce a denial-of-service attack. Whilst the logging system does not directly prevent this type of attack, it will enable system administrators to swiftly identify the problem and find a solution, allowing for availability to be restored as soon as possible.

Limitations & Trade-offs

Whilst these controls prevent certain attacks from occurring, they introduce trade-offs. For example, account lockout functionality mitigates the threat of brute-force-attacks by tracking

the username of the user who has attempted to log in. However, this allows attackers to abuse this system by intentionally entering incorrect passwords to prevent the user from logging in and gaining access to their own data, diminishing the availability of the system.

Another trade-off seen by the system is the choice to use SQLite. This is due to it using database-level locking during write operations. Whilst this is suitable for small-scale databases, this could reduce availability when under high-concurrency conditions. This highlights the importance of balancing security measures with maintaining reliable access, particularly where permanent availability is necessary.

5. Trade-off Analysis (CIA balance)

In practice, it is not possible to maximise confidentiality, integrity and availability without introducing trade-offs. For example, security controls designed to strengthen one aspect of the CIA triad can negatively impact the others, making balance a necessary consideration when developing systems.

Confidentiality vs Availability

Due to the security-focused nature of this system, large importance is placed upon confidentiality. This can be seen through measures taken such as note encryption which do prevent data from being exposed in the event of a data leak but also introduce more dependencies such as the encryption key which poses a threat to availability as if it is lost the notes can no longer be viewed. Another portion of the system where this can be seen is the lockout functionality of the system. This mitigates the threat of brute-force attacks but also introduces a new threat of easily implemented denial-of-service attacks which have the potential to impact availability.

Integrity vs Usability

Whilst controls such as access controls prevent users from manipulating other users' data. It requires very careful query design and increases system complexity. Similarly, the logging system allows for system administrators to detect potentially malicious activity and use this information to help maintain integrity by preventing these attacks early. However, this impact usability as the storing logs generates additional data that will need to be stored, managed and reviewed.

Security vs Performance / Simplicity

Firstly, decisions such as choosing SQLite will improve simplicity and portability. However, this limits performance through concurrency limits, potentially affecting availability if there are a high concurrency of requests to the database. This demonstrates that security often involves

balancing performance, complexity and risk rather than attempting to achieve absolute protection over the entire system.

6. Reflection & Real-World Relevance

I believe completing this project has enhanced my level of understanding of how security principles apply to real-world systems. The CIA triad analysis highlighted the importance of balancing confidentiality, integrity and availability based on risk and system requirements. This proved to me that it is unrealistic to have perfect end-to-end security, rather, that effective security design requires justified trade-offs instead of all-round protection.

Via the application of secure programming principles to a practical authentication system helped me bridge the gap between theoretical knowledge and real-world implementation. After analysing the system from a security perspective, I have further enhanced my skills in identifying weaknesses, evaluating the impact of the weaknesses, and proposing realistic solutions. This project has further provided a solid foundation for security-focused roles where risk management and system reliability are critical and reinforced my interest in Cyber-Security.